Software Architecture and Design

Chapter 1

Outline

- Concepts of S/W architecture
- Architectural Thinking
- Modularity
- Architecture Characteristics
- Measuring and Governing Architecture Characteristics
- Key Issues in Software Design
- Evolution of S/W Architecture

Concepts of s/w architecture

- Architecture is a *general* term to describe *buildings* and other *physical* structures.
- Architectures are not concerned with the creation of *building technologies* and *materials*.
- Architectures are concerned with how building materials can be *put together in desirable ways*.
- Architecture focus the Overall structure, organization, and high-level design. E.g. In software: Defining the major modules, In buildings: The overall layout, materials, and structural systems.
- Three things are important:
  - *Durability*: should last for a long time.
  - *Utility*: should be useful for the people,
  - *Beauty* : should look good and rise the users sprits.

Concepts of s/w architecture

- Software Engineering is a systematic approach to the development, operation and maintenance of software(an engineering-style system of software development)
- Software Model is a conceptual description and abstraction of software structures, components, interface and functions. software modeling is larger than an algorithm or a single method and ways of expressing a software design.
    - Architectural Model (design) and Development Model (methodology)
- Software Architecture is a high-level abstraction and description to the structure and organization of software system, also a blueprint for software development and evolution.
- Software Architectural Model is an expression of the viewpoint of software architecture, which is a rich, rigorous, and structural description on the software system's elements, functions and interface

Concepts of s/w architecture

Why S/W Architecture

Large-scale Software Systems

- number of lines of code (> million)
- number of people involved (> 100)
- amount of data stored, processed and manipulated
- number of hardware components
- heterogeneous platforms
- distributed (decentralized) environment

Concepts of s/w architecture

Why S/W Architecture

Meeting Key Aspects of Large-scale Software Systems

- Decentralization/Decomposition
- Importability
- Scalability
- Manageability
- Security
- Sustaining Engineering/Evolution

Concepts of s/w architecture

Why S/W Architecture

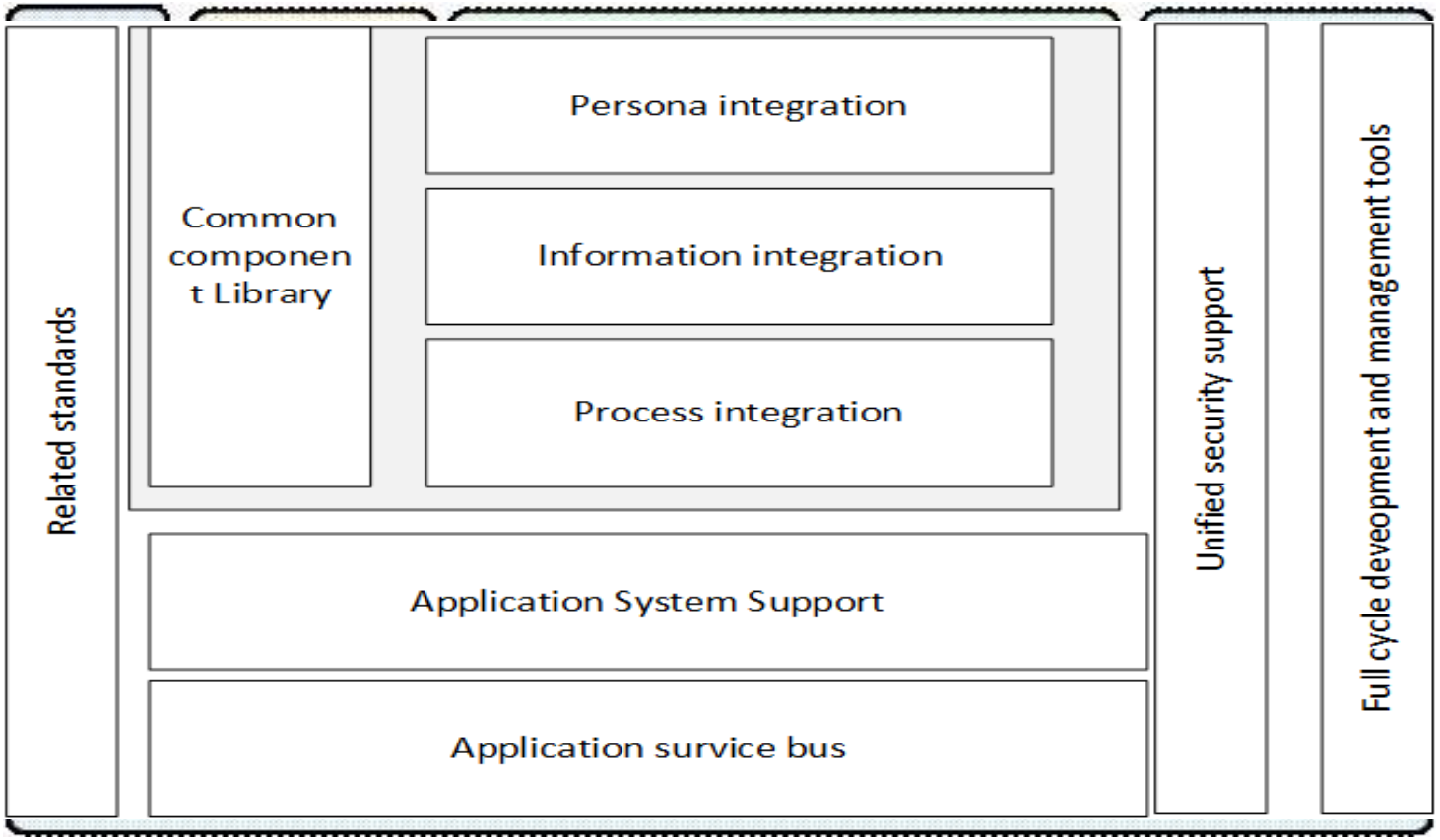
Large Application Domain

- Aerospace control/command system
- Satellite data processing system
- Global climate/environment modeling and monitoring system
- Complex combat applications
- Large-scale financial processing system
- Extremely high load web portals (such as Google’s distributed search engine)

Concepts of s/w architecture

Logic Architecture（Functional Architecture）

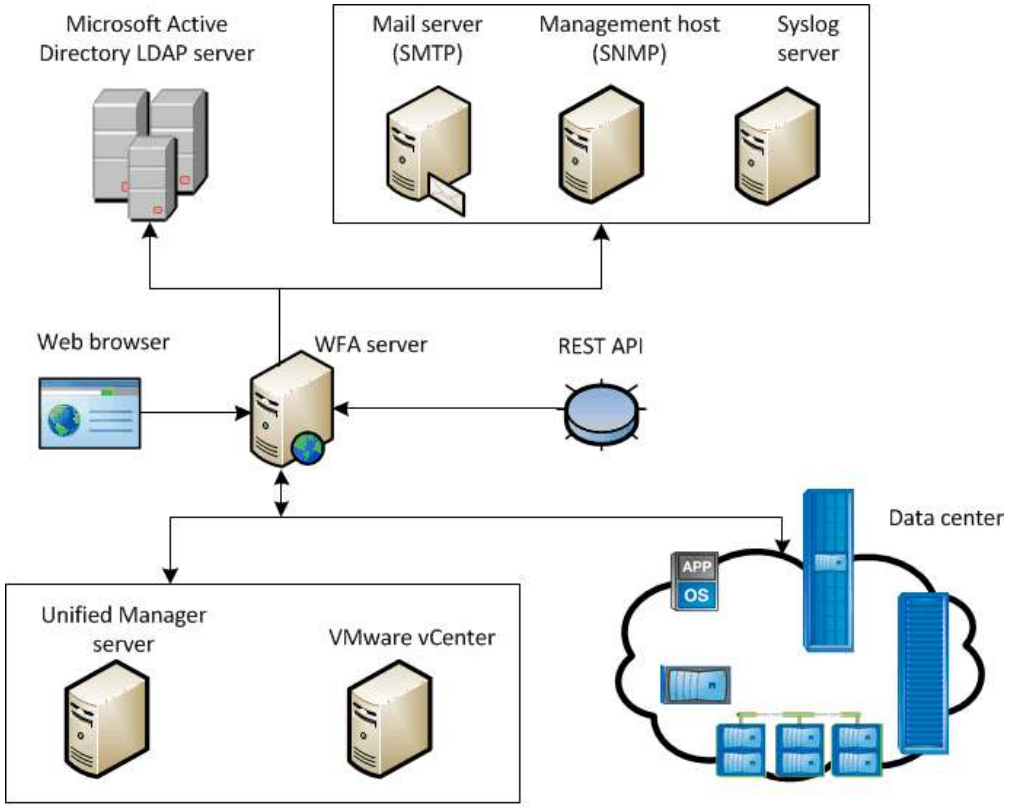
Description of logic/functional relationship among the software components, such as the UI, database, external system, business unit, etc.



Concepts of s/w architecture

Physical Architecture（System Architecture）

The mapping of software units to hardware nodes to describe the deployment of software systems in execution environment.

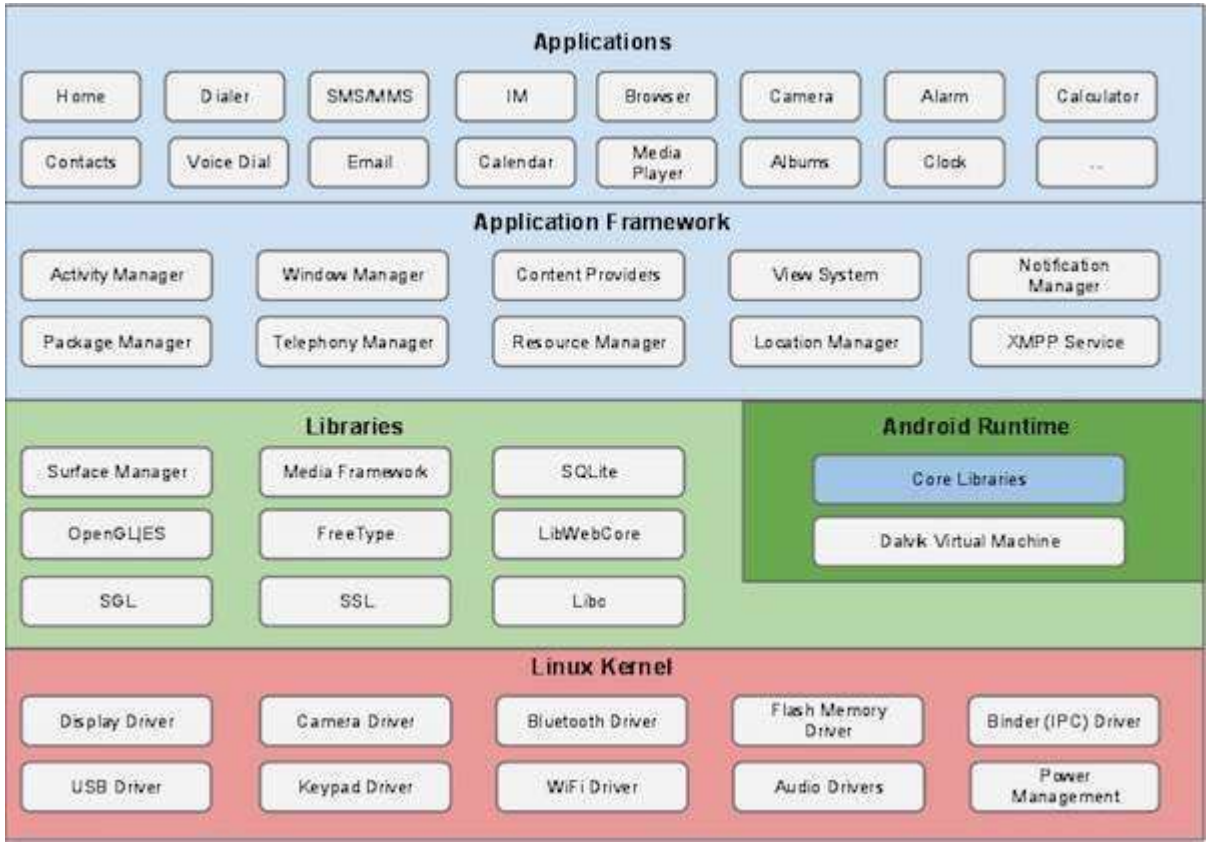


OnCommand Workflow Automation (WFA) deployment

Concepts of s/w architecture

Technological Architecture

Description of SDK, development technology, technological standards related to the design of software systems.



Concepts of s/w architecture

Software Design Process

- A process through which requirements are translated into a representation of software.
- The process of converting the system specification into an executable system.
- Design is about *how* to build a system.
- Design is the creative process of describing and transforming a problem into a solution.

From a project management point of view, software design can be conducted **in two main steps**:

1. **Preliminary Design**

   •Concerned with the transformation of requirements into data and software architecture.

2. Detail design

   •Focuses on refining the architectural representation, and lead to detailed data structure and algorithmic representations of software.

Concepts of s/w architecture

Software Design…

From a technical point of view, software design can be conducted **in two main steps**:

- **Architectural design**
- **Data design**
- **Procedural design**
- **Interface design**

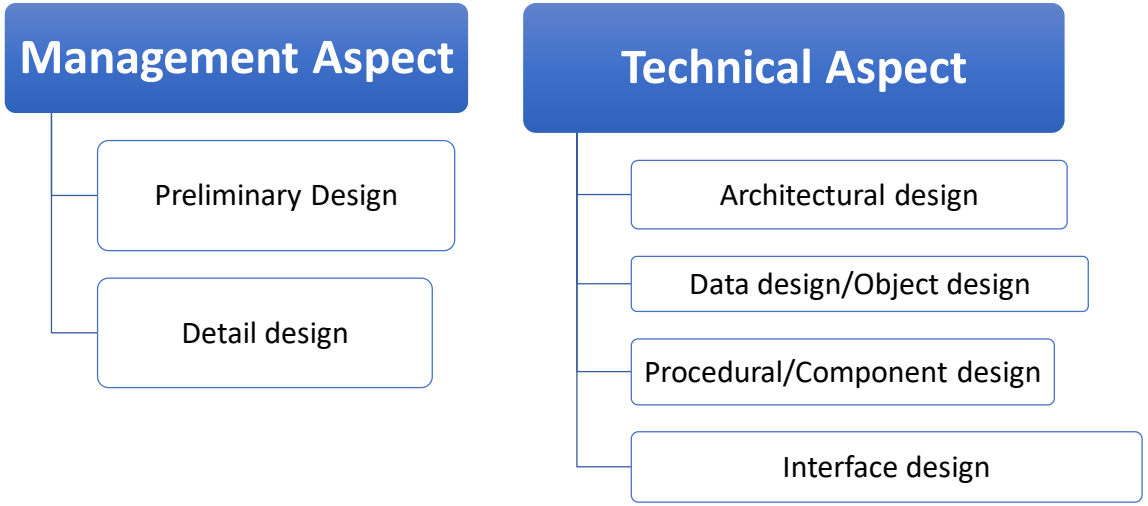
- **Architectural Design:** identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships and how they are distributed.
- **Data Design /Class design :**Objective of Data Design is to transform the information domain into data structures.

  Eg: Entity Relationship diagrams, Data dictionary, abstract data types and Data structures.

Concepts of s/w architecture

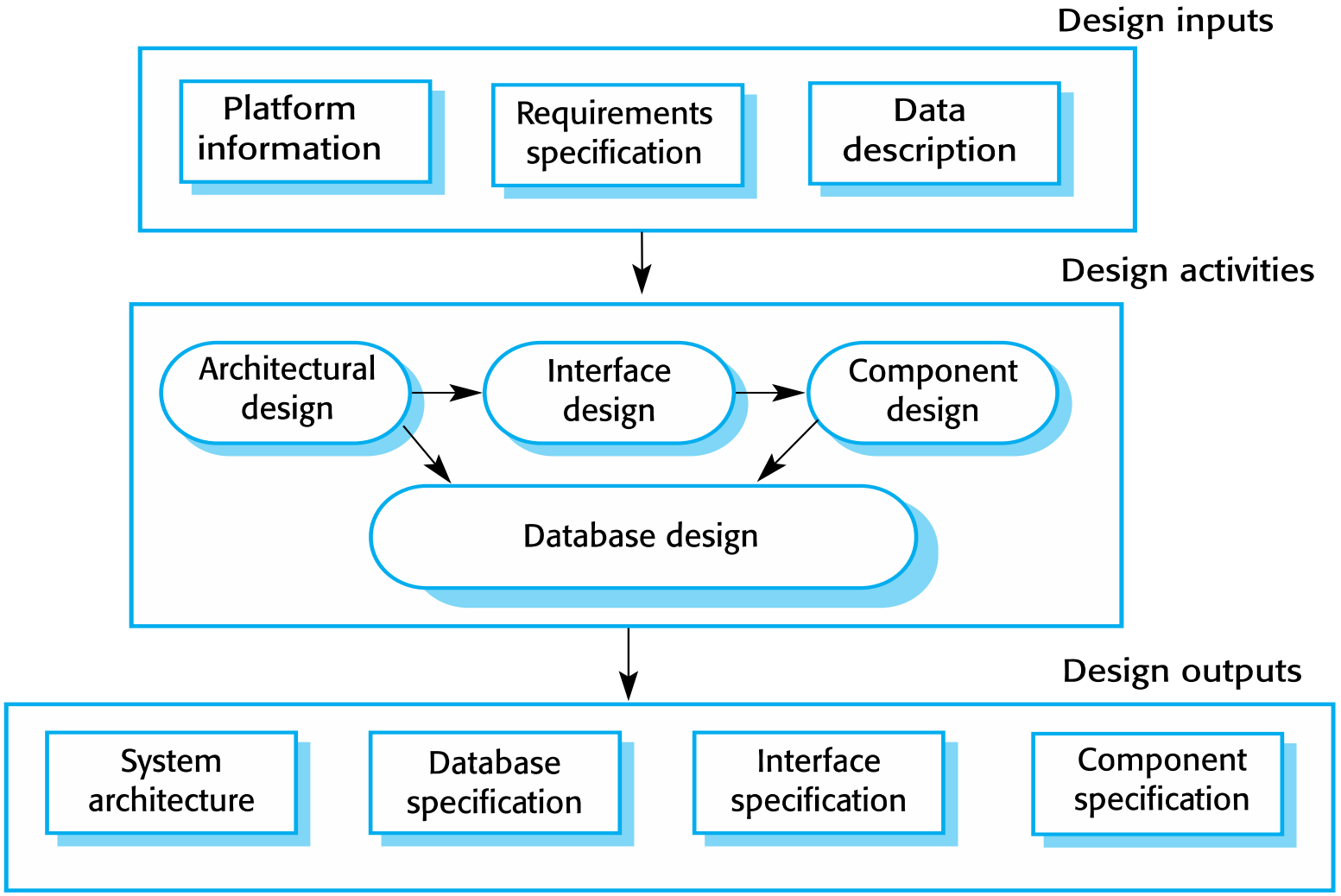
Software Design..

- **Procedural Design:** Objective is to transform structural components into a procedural/component description of the software. This step occurs after the data and program structures have been established, i.e. after architectural design.
- **Interface design:** establishes the layout and interaction mechanisms for human-machine interaction.



Concepts of s/w architecture

General model of the design process



Concepts of s/w architecture

Software Architecture	**Software Design**
- conceptual/logic abstraction	- plan/instruction
- skeleton/backbone of system	- blueprint of components
- common/general attributes of a set of architectural designs	- descriptions of structure and function of s/w
- what happen if 1000 users press one button	- what happen if one user presses one button
- provides a way to predict or judge system performance	- provides a way to assure system functions
- Principles	- Solutions

Architectural Thinking

There are four main aspects of thinking like an architect

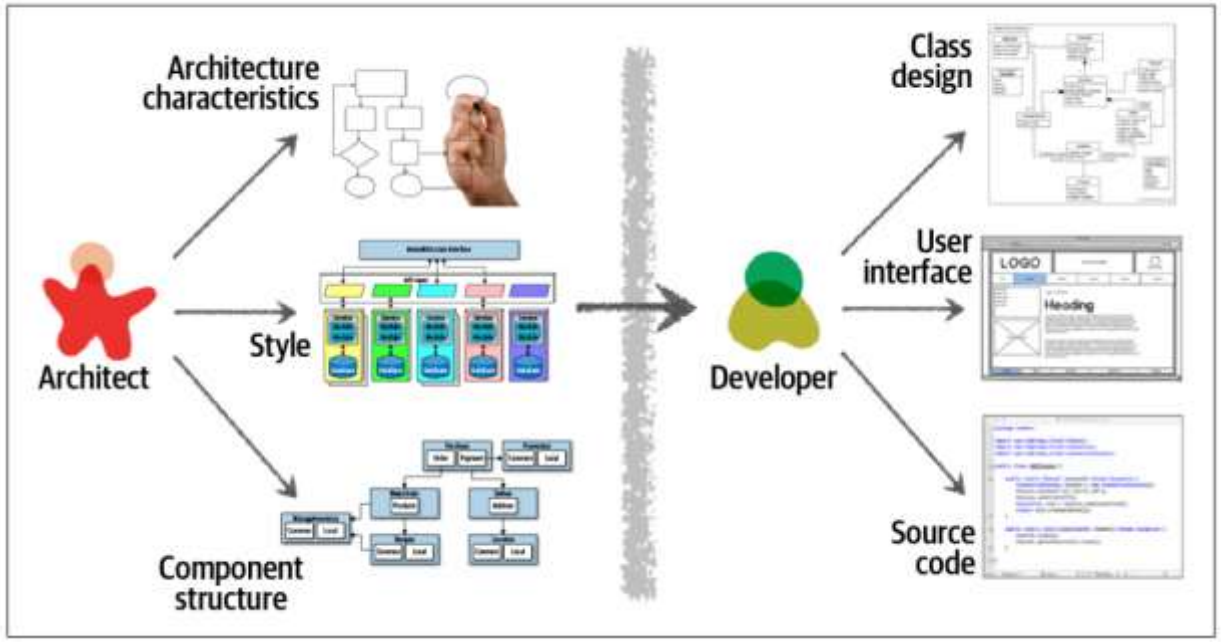
1. It's understanding the difference between **architecture and design** and knowing how to collaborate with development teams to make architecture work.
2. It's about **having a wide breadth of technical knowledge** while still maintaining a certain level of technical depth, allowing the architect to see solutions and possibilities that others do not see.
3. It's about **understanding, analyzing, and reconciling trade-offs** **b**etween various solutions and technologies.
4. It's about **understanding the importance of business drivers** and how they translate to architectural concerns.

Architectural Thinking

Architecture Versus Design

Where does architecture end and design begin?

- The architect is disconnected from the development teams
- Decisions an architect makes sometimes never make it to the development teams, and decisions development teams make that change the architecture rarely get back to the architect.



Traditional view of architecture versus design

- To make architecture work, Both the physical and virtual barriers between architects and developers must be broken down.
- Thus, forming a strong bidirectional relationship between architects and development teams.

Architectural Thinking

Architecture Versus Design

An architect is responsible for things like:

- Analyzing business requirements to extract and define the architectural characteristics
- Selecting which architecture patterns and styles would fit the problem domain, and
- Creating components (the building blocks of the system).

The artifacts created from these activities are then handed off to the development team.

The Developer is responsible for things like:

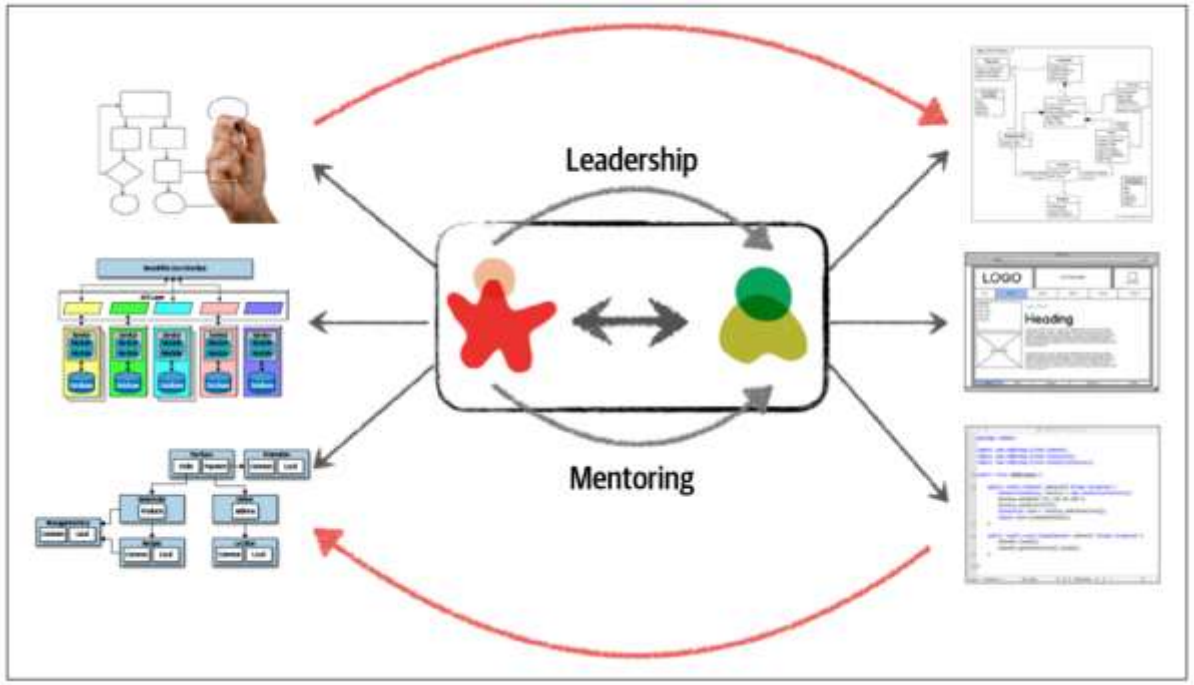
- Creating class diagrams for each component,
- Creating user interface screens, and
- Developing and testing source code.

Architectural Thinking

Architecture Versus Design

Where does architecture end and design begin?

- A tight collaboration between the architect and the development team is essential for the success of any software project.
- **So where does architecture end and design begin? It doesn't.**



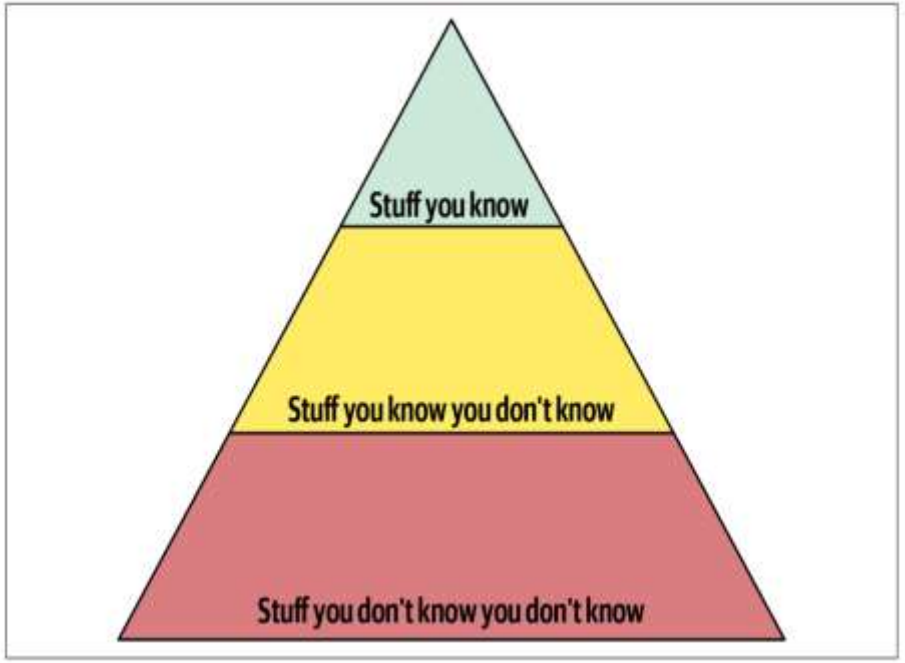
Making architecture work through collaboration

- They are both part of the circle of life within a software project and kept in synchronization with each other in order to succeed.

Architectural Thinking

Technical Breadth

- **Stuff you know** includes the technologies, frameworks, languages, and tools a technologist uses on a daily basis to perform their job, such as knowing Java as a Java programmer.
- **Stuff you know you don't know** includes those things a technologist knows a little about or has heard of but has little or no expertise in.



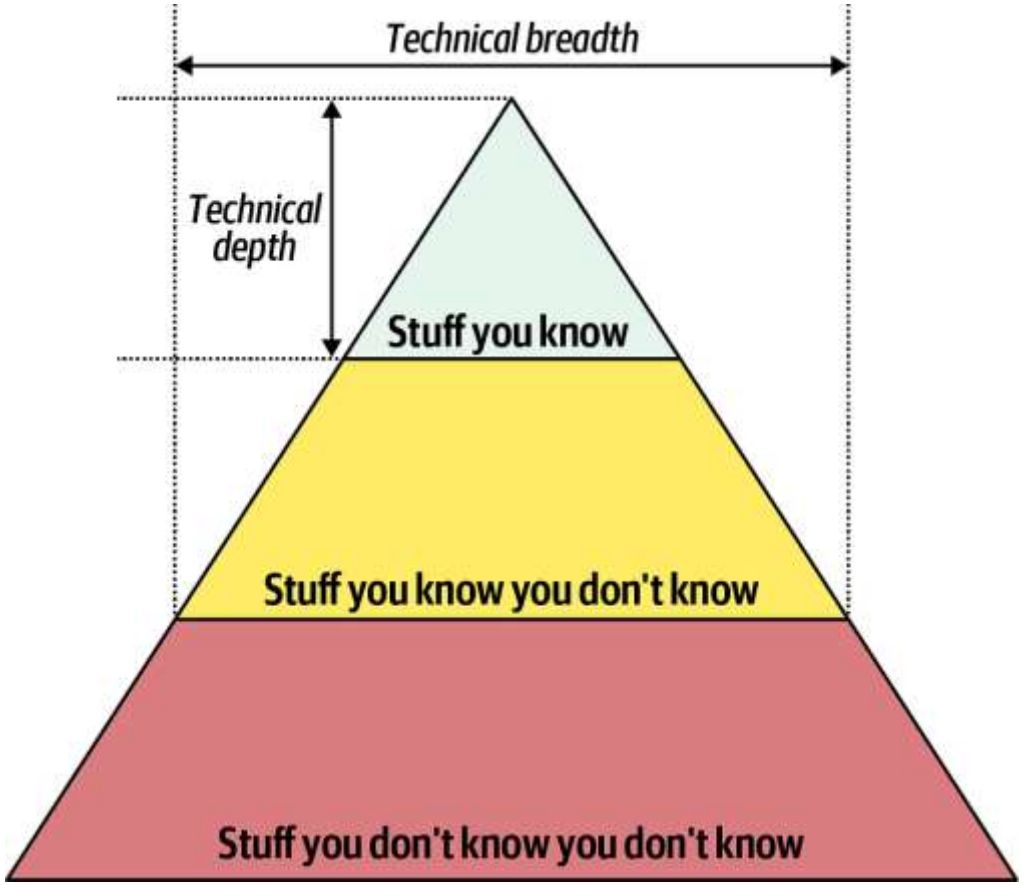
The pyramid representing all knowledge

- **Stuff you don't know you don't know** is the largest part of the knowledge triangle and includes the entire host of technologies, tools, frameworks, and languages that would be the perfect solution to a problem a technologist is trying to solve, but the technologist doesn't even know those things exist.

Architectural Thinking

Technical depth

- As a developer the stuff you know is also the stuff you must maintain.
  - *You have good practical skills and knowledge in a specific area or activity*
- Nothing is static in the software world.

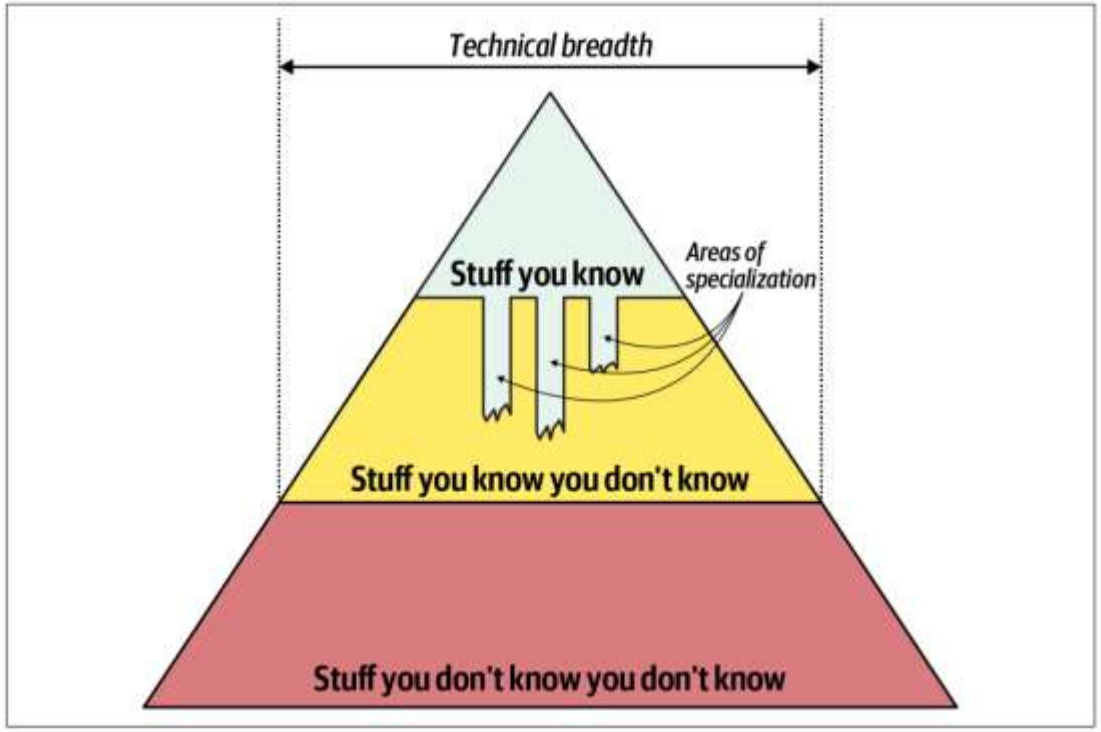


Developers *must maintain expertise to retain it*

Architectural Thinking

Technical breadth

- As an architect, breadth is more important than depth
- Because architects must make decisions that match capabilities to technical constraints, a broad understanding of a wide variety of solutions is valuable.
- Developers leading to architect's role must focus on breadth instead of expertise (depth).



*Enhanced breadth and shrinking depth for the **architect** role*

Architectural Thinking

Analyzing Trade-Off

- Thinking like an architect is all about seeing trade-offs in every solution, technical or otherwise.
- We cannot Google the answer to whether REST(synchronous communication) or messaging (asynchronous communication) would be better, or whether micro services is the right architecture style, because it does depend.
- It depends on the
  - Deployment environment,
  - business drivers,
  - company culture,
  - budgets,
  - timeframes,
  - Developer skill set, and
  - dozens of other factors.

"There are no right or wrong answers in architecture—only trade-offs."

Analyzing Trade-Off

Some common trade-offs encountered in software development:

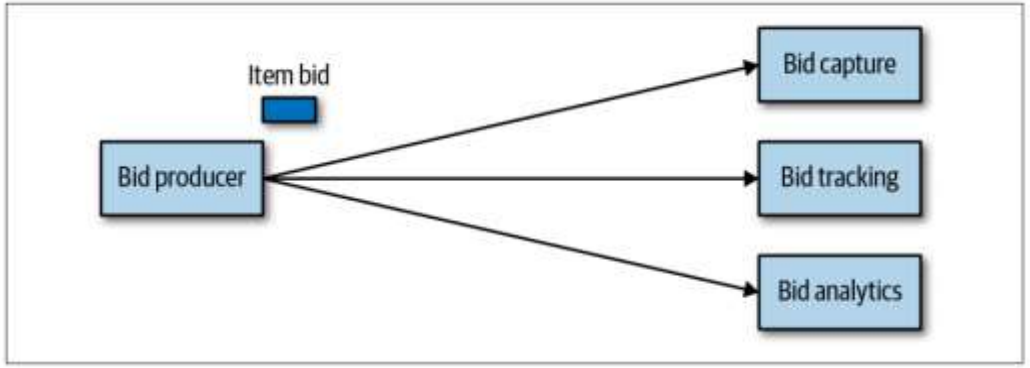
- **Performance vs. Scalability:** Optimizing for speed and efficiency for individual users vs. designing the system to handle increased load and data volume.
- **Cost vs. Maintainability:** Minimizing initial development expenses vs. ensuring long-term ease of updates and bug fixes,
- **Development Speed vs. Code Quality:** Delivering features quickly vs. writing robust, reliable, and maintainable code.
- **User Requirements vs. Technical Feasibility:** Meeting user needs and desired functionality vs. considering the limitations of available technologies and resources.
- **Performance vs. Resource Utilization:** Optimizing for speed and efficiency vs. managing the consumption of resources like memory and processing power.

Understanding these trade-offs and finding the optimal balance is crucial for successful software delivery.

Example Analyzing Trade-Off

- For example, consider an item auction system, where someone places a bid for auction.

- The Bid Producer service generates a bid from the bidder and then sends that bid amount to the **Bid Capture**, **Bid Tracking**, and **Bid Analytics services**.
- This could be done by using queues in a **point-to-point messaging** or by using a **topic in a publish-and-subscribe messaging** fashion.



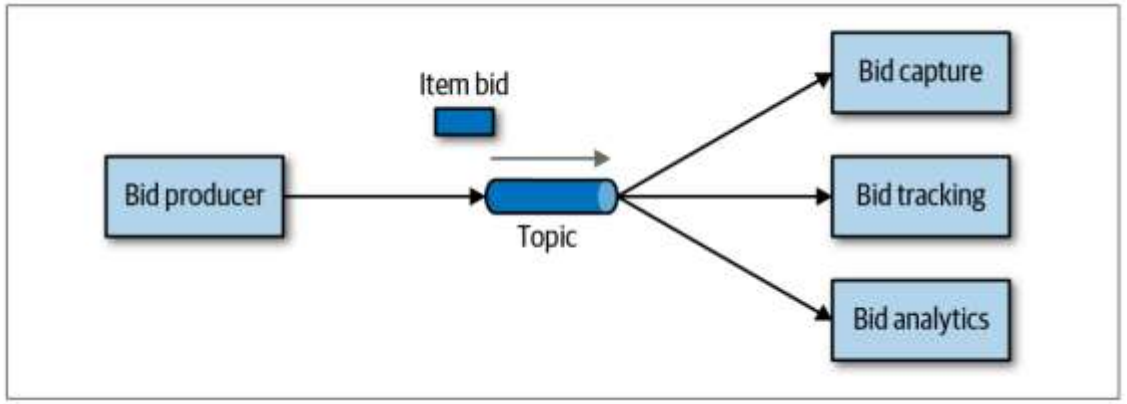
Auction system example of a trade-off ***queues*** *or* ***topics****?*

- Which one should the architect use? Architectural thinking requires the architect to analyze the trade-offs associated with each option and select the best one given the specific situation.

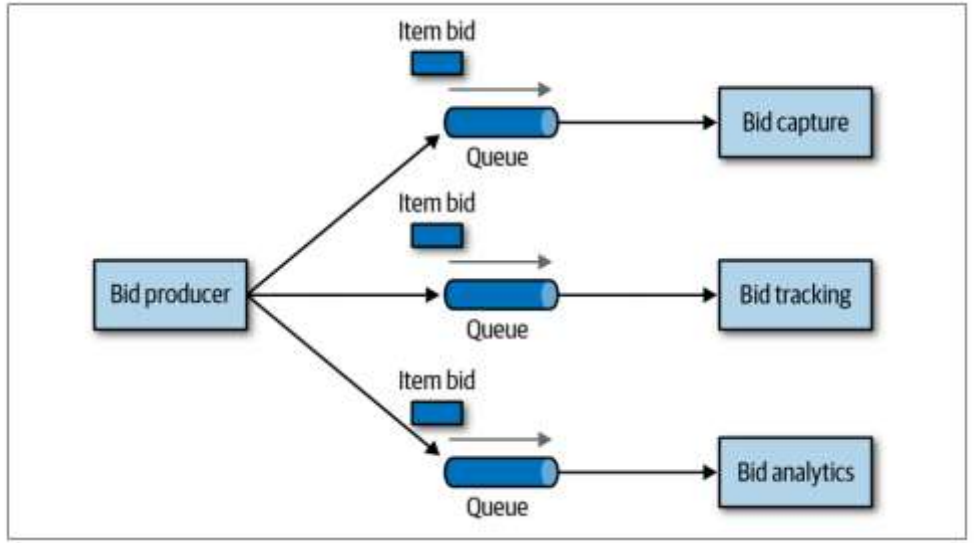
Architectural Thinking

Example Analyzing Trade-Off

- For example, consider an item auction system, where someone places a bid for auction.



Use of a topic for communication between services



Use of queues for communication between services

- Which is the better option? And the answer? It depends!

Architectural Thinking

Example Analyzing Trade-Off

- Which is more important: extensibility or security?”

summarize these trade-offs on using the topic:

Topic advantages	Topic disadvantages
Architectural extensibility	Data access and data security concerns
Service decoupling	No heterogeneous contracts
	Monitoring and programmatic scalability

- Using **queue** requires significant change to the system when adding new bidding functionality.
- Using **queue** Bid producer and Bid Services are tightly coupled
- The decision between different solutions will always depend on the business drivers, environment, and a host of other factors.

Modularity

- **Modularity** refers to each of a set of standardized parts or independent units that can be used to construct a more complex structure.
- We use modularity to describe a logical grouping of related code, such as classes.
- Most languages provide mechanisms for modularity (package in Java, namespace in .NET,and so on)
- **com.mycompany.customer** package in Java should contain things related to customers.
- Architects must be aware of how developers package things because it has important implications in architecture.
- For example, if several packages are tightly coupled together, reusing one of them for related work becomes more difficult.

Modularity

Measuring Modularity

The three key concepts:

- ***Cohesion***
- ***Coupling* and**
- ***Connascence***

> Attempting to divide a cohesive module would only result in increased coupling and decreased readability.
> —Larry Constantine

Cohesion

- Refers to the degree to which the elements within a module are related to each other.
- In a cohesion all the parts should be packaged together, because breaking them into smaller pieces would require coupling the parts together via calls between modules to achieve useful results.

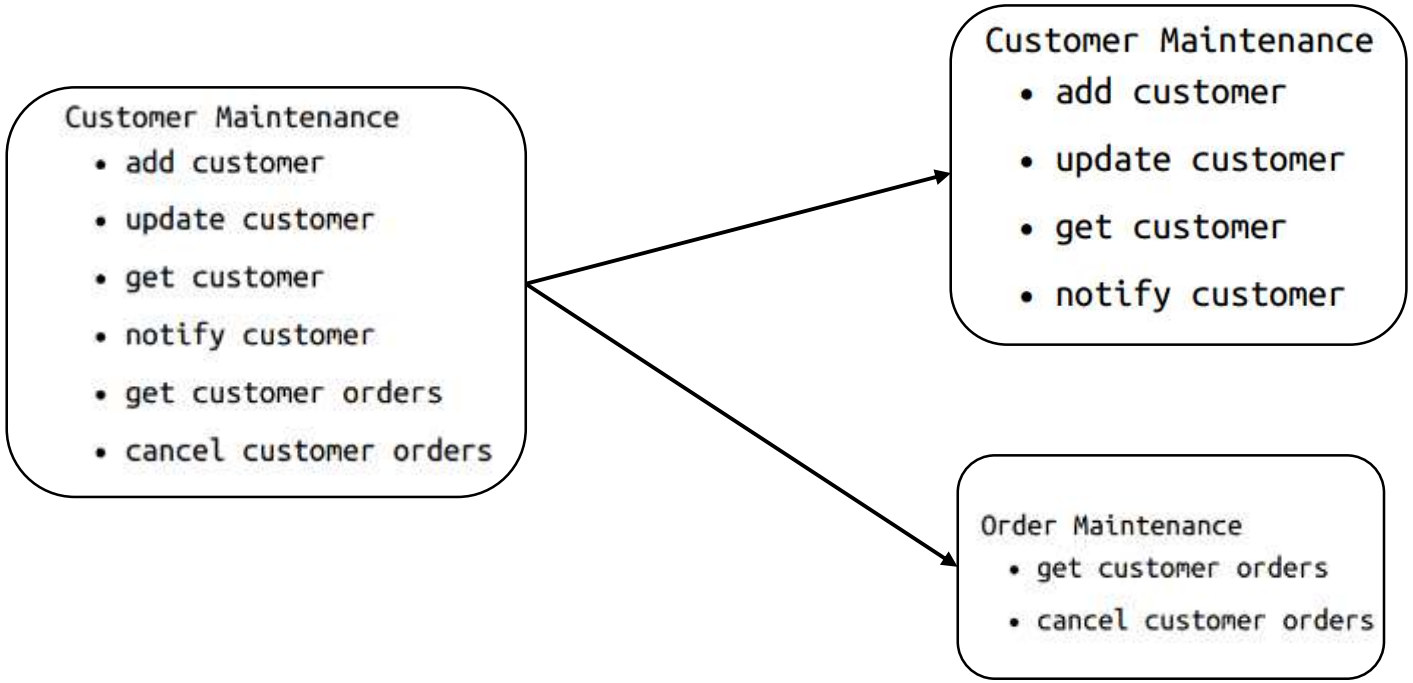
Modularity

Types of Cohesion

- Functional cohesion: Every part of the module is related to the other, and the module contains every-thing essential to function. Best design method.
- Sequential cohesion: Two modules interact, where one outputs data that becomes the input for the other
- Communicational cohesion: Two modules form a communication chain, where each operates on information and/or contributes to some output. For example, add a record to the database and generate an email based on that information.
- Procedural cohesion: Two modules must execute code in a particular order.
- Temporal cohesion: Modules are related based on timing dependencies. For example, many systems have a list of seemingly unrelated things that must be initialized at system startup; these different tasks are temporally cohesive…. etc .

Modularity

Consider this module definition:



Measuring Modularity

Which is the correct structure? As always, it depends:

- Are those the only two operations for Order Maintenance? If so, it may make sense to collapse those operations back into Customer Maintenance
- Is Customer Maintenance expected to grow much larger, encouraging developers to look for opportunities to extract behavior
- Does Order Maintenance require so much knowledge of Customer information that separating the two modules would require a high degree of coupling to make it functional?

Trade-off analysis of a software architect.

Related code should be close to each other to make it highly cohesive.

Modularity

Measuring Modularity

The Chidamber and Kemerer, **Lack of Cohesion in Methods** (LCOM) metric measures the structural cohesion of a module,

- In Class X, the LCOM score is low, indicating good structural cohesion.
- Class Y, lacks cohesion; each of the field/method pairs in Class Y could appear in its own class without affecting behavior.
- Class Z shows mixed cohesion, were developers could refactor the last field/method combination into its own class.

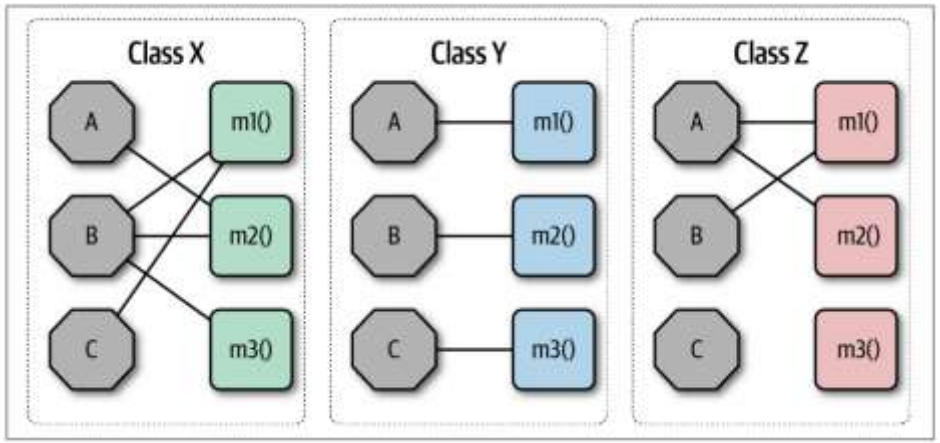


Illustration of the LCOM metric, where fields are octagons and methods are squares

Modularity

Measuring Modularity

Coupling

- Describes the degree to which modules are dependent on each other
- Degree of interaction between two modules. Five levels of coupling
  - **Data coupling:** two modules have write access to the same global data (best).
  - **Stamp coupling:** data structure is passed as parameter, but called module operates on only some of individual components
  - **Control coupling:** one module passes an element of control to the other.
  - **Common coupling:** Modules share a common data area, like global variables or a shared data structure.
  - **Content coupling :**one module directly references contents of the other (worst)
    - It occurs when one module directly relies on the *internal details* of another module

Modularity

Measuring Modularity

Coupling e.g.

- **(Data Coupling, Message Coupling): Good Coupling**

You order food from a restaurant using a menu (the interface). You don't care how the chef prepares it, as long as you get the dish you ordered.

- **Content Coupling: Bad Coupling**

You go into the restaurant's kitchen and tell the chef exactly how to cook the dish, maybe even rearranging their ingredients and tools. If the chef changes their cooking process or the kitchen layout, you're in trouble!

Modularity

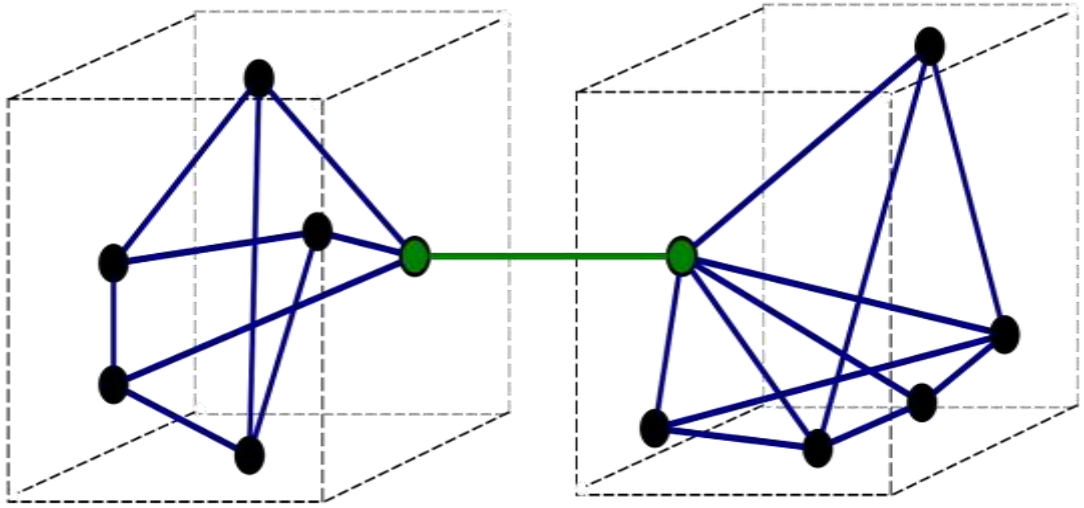
Coupling

- Relationship between two or more modules
- Focuses on measuring a module's dependency with other modules
- It is ideal to have low coupled modules which states dependency between modules
- It is based on communication between two modules. Therefore, it is known as **inter-module concept**
- There are 6 types of coupling: Data Coupling, Stamp Coupling, Control Coupling, External Coupling, Common Coupling and Content Coupling
- It is not possible to achieve no coupling among all modules

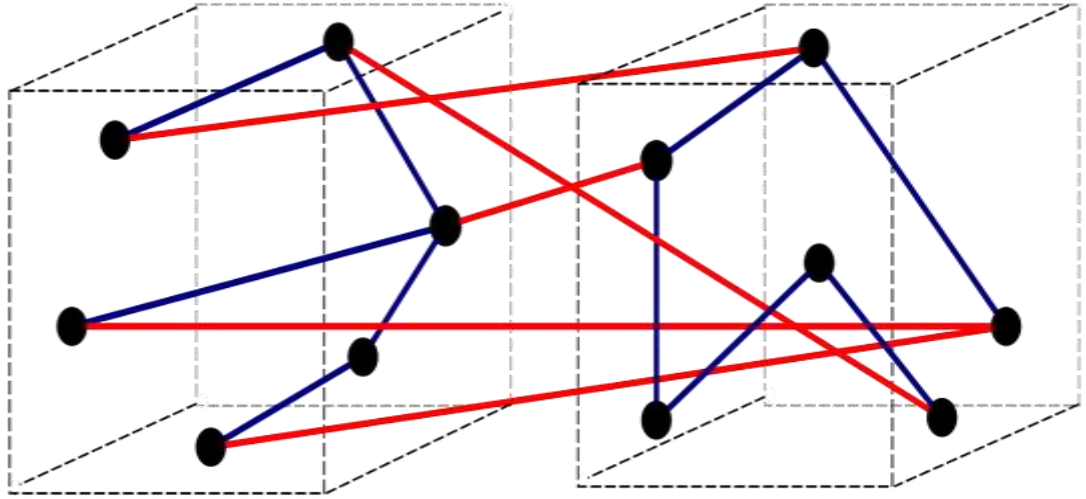
Cohesion

- It refers to relationship between two or more elements within a module
- It focuses on measuring a module's functional strength
- It is ideal to have a module with high cohesiveness which states implementation of a particular feature with low or no communication with other module
- It is based on communication between two elements or fields within a single module. Therefore, it is known as intra-module concept
- There are 8 types of cohesion: Functional Cohesion, Sequential Cohesion, Communicational Cohesion, Procedural Cohesion, Temporal Cohesion, Logical Cohesion and Coincidental Cohesion
- It is possible to achieve full cohesive modules

Modularity



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

- Multiple modules/packages/components that are **highly coupled** are strongly dependent on each other.
- Multiple modules/packages/components that are **loosely coupled** are not or somehow dependent on each other.
- For low coupled classes, changing something major in one class should not affect the other.
- *High coupling would make it difficult to change and maintain your code*
- Cohesion should always be high means that a module/package/component is *focused on what it should be doing*

Modularity

Measuring Modularity

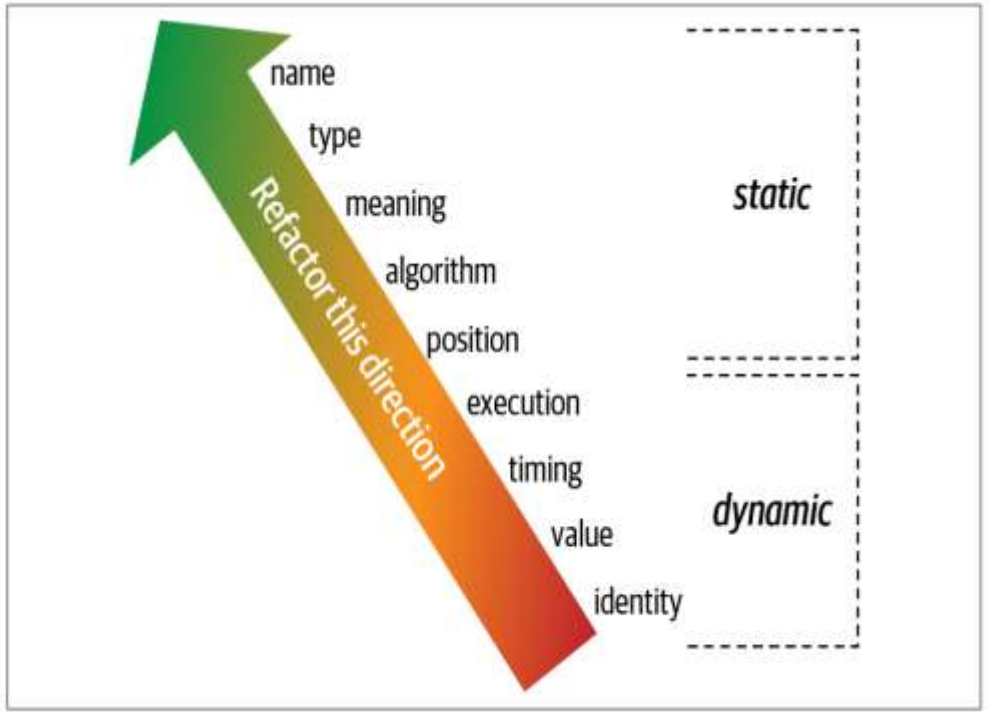
Connascence

- Describes the degree to which two or more components must agree about something.
- Closely related to cohesion, focuses on the visibility of implementation details within a module.

Two components are connascent if a change in one would require the other to be modified in order to maintain the overall correctness of the system. —Meilir Page-Jones

He developed two types of connascence: : static and dynamic.

- *Static connascence:* refers to source-code-level coupling.
- Dynamic connascence: analyzes calls at runtime.



The strength on connascence provides a good refactoring guide

Modularity

Measuring Modularity

Types of Static Connascence

- Name: Multiple components must agree on the name of an entity
- Type: Multiple components must agree on the type of an entity
- Meaning: Multiple components must agree on the meaning of particular values
- Position: Multiple components must agree on the order of values
- Algorithm: Multiple components must agree on a particular algorithm

Types of Dynamic connascence

- Execution: The order of execution of multiple components is important
- Timing: The timing of the execution of multiple components is important.
- Values: Occurs when several values relate on one another and must change together
- Identity: multiple elements in a system refer to the same logical entity

Imagine a system with two modules:

- **Module A:** Displays a customer's name and ID on a report.
- **Module B:** Sends marketing emails to customers based on their IDs.

Architecture Characteristics Defined

- An architecture characteristic meets three criteria:
  - Specifies a non domain design consideration (Aspects of software design that are *not* directly related to the core business logic or functionality)
  - Influences some structural aspect of the design (architectural impact on the design)
  - Is critical to application success
- Architecture characteristics ranging from low-level code characteristics, such as modularity, to sophisticated operational architecture characteristics, such as performance, scalability, elasticity, availability, and reliability.

Architecture Characteristics Defined

Operational Architecture Characteristics

Describe how a system functions and behaves in its runtime environment.

Term	Definition
Availability	How long the system will need to be available (if 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure).
Continuity	Disaster recovery capability.
Performance	Includes stress testing, peak analysis, analysis of the frequency of functions used, capacity required, and response times. Performance acceptance sometimes requires an exercise of its own, taking months to complete.
Recoverability	Business continuity requirements (e.g., in case of a disaster, how quickly is the system required to be on-line again?). This will affect the backup strategy and requirements for duplicated hardware.
Reliability/ safety	Assess if the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money?
Robustness	Ability to handle error and boundary conditions while running if the internet connection goes down or if there's a power outage or hardware failure.
Scalability	Ability for the system to perform and operate as the number of users or requests increases.

Common operational architecture characteristics

Architecture Characteristics Defined

Structural Architecture Characteristics: *Define the basic structure of a software system*

Term	Definition
Configurability	Ability for the end users to easily change aspects of the software's configuration (through usable interfaces).
Extensibility	How important it is to plug new pieces of functionality in.
Installability	Ease of system installation on all necessary platforms.
Leverageability/ reuse	Ability to leverage common components across multiple products.
Localization	Support for multiple languages on entry/query screens in data fields; on reports, multibyte character requirements and units of measure or currencies.
Maintainability	How easy it is to apply changes and enhance the system?
Portability	Does the system need to run on more than one platform? (For example, does the frontend need to run against Oracle as well as SAP DB?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Upgradeability	Ability to easily/quickly upgrade from a previous version of this application/solution to a newer version on servers and clients.

Structural architecture characteristics

Architecture Characteristics Defined

Cross-Cutting Architecture Characteristics: *Functionalities that affect the entire application*

Term	Definition
Accessibility	Access to all your users, including those with disabilities like colorblindness or hearing loss.
Archivability	Will the data need to be archived or deleted after a period of time? (For example, customer accounts are to be deleted after three months or marked as obsolete and archived to a secondary database for future access.)
Authentication	Security requirements to ensure users are who they say they are.
Authorization	Security requirements to ensure users can access only certain functions within the application (by use case, subsystem, webpage, business rule, field level, etc.).
Legal	What legislative constraints is the system operating in (data protection, Sarbanes Oxley, GDPR, etc.)? What reservation rights does the company require? Any regulations regarding the way the application is to be built or deployed?
Privacy	Ability to hide transactions from internal company employees (encrypted transactions so even DBAs and network architects cannot see them).
Security	Does the data need to be encrypted in the database? Encrypted for network communication between internal systems? What type of authentication needs to be in place for remote user access?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Usability/ achievability	Level of training required for users to achieve their goals with the application/solution. Usability requirements need to be treated as seriously as any other architectural issue.

Cross-cutting architecture characteristics

Architecture Characteristics Defined

Identifying Architectural Characteristics

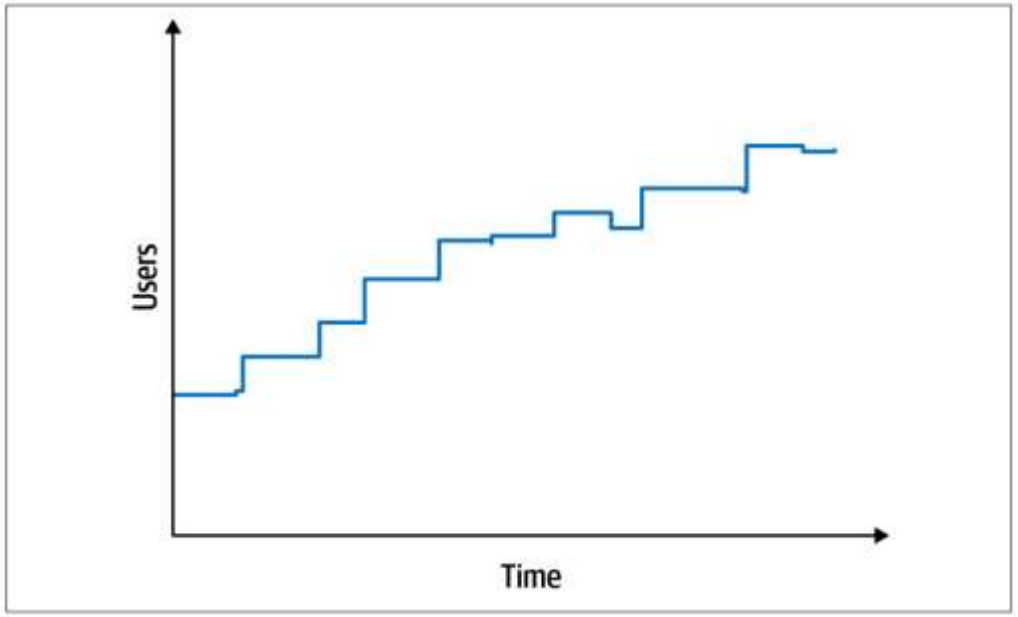
- An architect uncovers **architecture characteristics** in at least three ways by
  1. Extracting from domain concerns
  2. Extracting from Requirements and
  3. Extracting from Implicit domain knowledge. *Many architecture characteristics aren't specified in requirements documents. E.g the system might want to support is Availability*

Extracting Architecture Characteristics from Domain Concerns

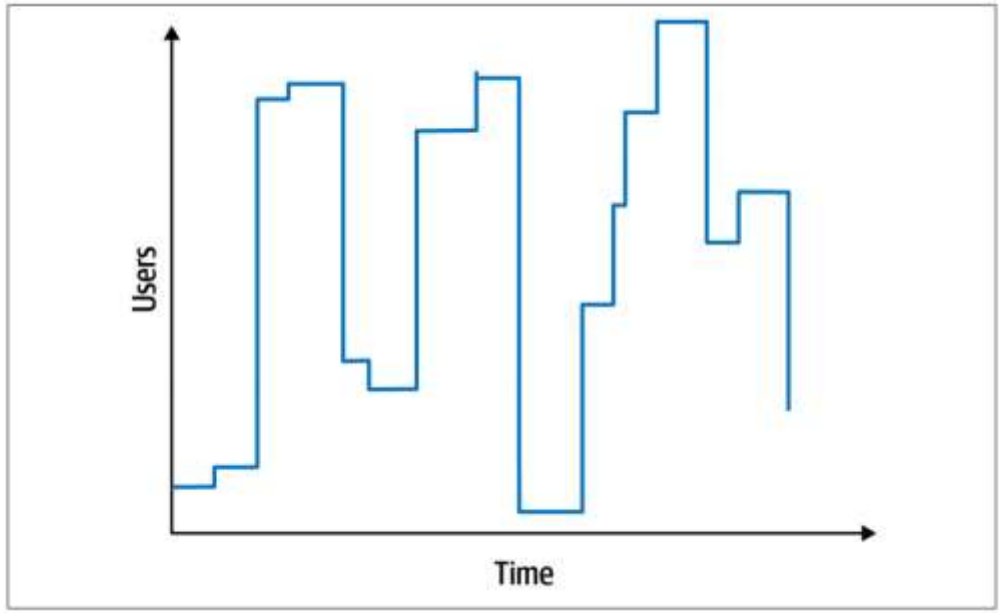
Domain concern	Architecture characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

Architecture Characteristics Defined

Extracting Architecture Characteristics from Requirements, For example, explicit expected numbers of users



Scalability measures the performance of concurrent users



Elastic systems must withstand bursts of users

- ***Scalability*** —the ability to handle a large number of concurrent users without serious performance degradation.
- ***Elasticity***— the ability to handle bursts of requests (The need for systems to be responsive, stable, and scalable in the face of fluctuating user traffic).

Measuring and Governing Architecture Characteristics

Measuring Architecture Characteristics

Operational Measures

- Many architecture characteristics have obvious direct measurements, such as performance or scalability.

Structural Measures

- What about internal structural characteristics, such as well-defined modularity? Unfortunately, comprehensive metrics for internal code quality don’t yet exist.

Process Measures

- Some architecture characteristics intersect with software development processes. For example, agility often appears as a desirable feature. However, it is a composite architecture characteristic that architects may decompose into features such as testability, and deploy ability.

Measuring and Governing Architecture Characteristics

Governance and Fitness Functions

Governing Architecture Characteristics

- The scope of architecture governance covers any aspect of the software development process require architectural governance
  - Coping incremental growth in capabilities within the software development ecosystem
  - Continuous integration via Extreme Programming created, which led to further automation into operations (DevOps)

Architecture fitness function

- Any mechanism that provides an objective integrity assessment of some architecture characteristic or combination of architecture characteristics.

Scope of Architecture Characteristics

- Software architecture traditionally placed the scope of architecture characteristics at the **system level.**
- For example, scalability, refers scalability of the entire system. That was a safe assumption a decade ago, when virtually all systems were monolithic.
- With the advent of modern engineering techniques and the architecture styles they enabled, such as micro services, the scope of architecture characteristics has narrowed considerably.
- When evaluating many operational architecture characteristics, an architect must consider dependent components outside the code base that will impact those characteristics.

Key Issues in Software Design

A number of key issues are quality concerns that all software must such as, performance, security, reliability, usability, etc.

- ***Concurrency*** is concerned with decomposing software into processes, tasks, and threads and dealing with related issues of efficiency, atomicity, synchronization, and scheduling.
- ***Control and Handling of Events*** is concerned with how to organize data and control flow as well as how to handle reactive and temporal events through various mechanisms such as implicit invocation and call-backs.
- ***Data Persistence*** This design issue is concerned with how to handle long-lived data.
- ***Distribution of Components*** is concerned with how to distribute the software across the hardware, how the components communicate, and how middleware can be used to deal with heterogeneous software.

Key Issues in Software Design

- ***Error and Exception Handling and Fault Tolerance*** This design issue is concerned with how to prevent, tolerate, and process errors and deal with exceptional conditions.
- ***Interaction and Presentation*** is concerned with how to structure and organize interactions with users as well as the presentation of information (for example, separation of presentation and business logic using the Model-View-Controller approach).
- ***Security*** prevent unauthorized disclosure, creation, change, deletion, or denial of access to information and other resources.

Evolution of S/W Architecture

Evolution History

- Early days: a series of execution codes that use computing logics and algorithms from the program start to the end
- Then people found, to the large-scale s/w systems, the architectural design is more important than data structure and algorithm
- Experienced programmers tend to use certain types of architectural designs, yet no models or design standards that can follow

Evolution of S/W Architecture

1, No-architecture Stage（1946～1969）

- Programming languages: machine code and assembly language, emerge of advanced languages such as FORTRAN and COBOL
- Programs were small and simple, without considerations on s/w architecture
- Hardware system was simple, too, and s/w systems are mainly centralized
- S/W was bonded to h/w platform without importability

Evolution of S/W Architecture

2, Infant Stage (1970～1980)

- Majority were advanced programming languages
- Software Engineering was initiated and researched， and development models, methodology and SDKs were developed
- The preliminary design and detailed design were conducted to guide building a s/w system, based on the schemes of control flow and data flow

Evolution of S/W Architecture

3, Junior Stage(1980～1990)

- Appearance of large-scale s/w systems and the object-oriented technology, which allowed the designer, developer and user to more precisely present their design work, which greatly improved the quality of s/w products
- In 1994， Booch、Rumbaugh、Jacobson invented the Unified Modeling Language (UML)
- S/W architectural design draw attentions

Evolution of S/W Architecture

4, Advanced Stage (since 1990)

- S/W systems become more and more complex and larger scale
- Algorithm + Data Structure converted to Component + Architectural Assembly
- Advanced abstraction: decouple Arch. Model from Arch. Design (Kruchten's “4＋1”Model)
- No unified definition, no widely accepted standards yet